

SNN-PointNet: Technical Q&A

Detailed Answers to Implementation Questions
(Companion to the SNN-PointNet Experimental Report)

Purdue Project — SNN-PointNet Experiment

March 13, 2026

Contents

1	How is Mean Exit Calculated?	2
1.1	Step-by-Step Procedure	2
1.2	Code Implementation	2
1.3	Interpretation	2
2	How is the Efficiency Proxy Calculated?	3
2.1	Physical Motivation	3
2.2	Formula	3
2.3	Code Implementation	3
2.4	Worked Example	3
3	How and Why Did Training Stop?	4
3.1	Intended Training Schedule	4
3.2	Why It Crashed (E3DSNN Model)	4
3.3	The Fix (TBPTT)	4
4	What is the Total Spike Rate?	5
4.1	Definition	5
4.2	How It Is Tracked	5
4.3	Significance	5
5	What is the CDF in the Early-Exit Context?	5
5.1	Definition	6
5.2	Interpretation	6
5.3	Why It Matters	6
6	What Does the Confidence Threshold Mean?	6
6.1	Formal Definition	7
6.2	Practical Meaning	7
6.3	Threshold Sweep Results	7
6.4	How to Choose θ	7
6.5	Connection to the Auxiliary Loss	7
7	Why Does ANN Accuracy Appear Higher in the Graph Even Though SNN Is Claimed to Be Competitive?	7
7.1	What the Graph Actually Shows	8
7.2	Why SNNs Converge Slower	8

7.3	What Would the Graph Look Like After Full Training?	8
7.4	Why the ANN Slice Curve Is <i>Flat</i>	8
7.5	Summary	9

1 How is Mean Exit Calculated?

Short answer: Mean exit is the average timestep at which the model’s confidence in its top-1 prediction first crosses the threshold θ , averaged across all validation samples.

1.1 Step-by-Step Procedure

During evaluation, the model processes each point cloud slice-by-slice ($t = 1, 2, \dots, T$, where $T = 16$). At every timestep, logits are passed through `softmax` to yield a probability vector $\mathbf{p}_t \in \mathbb{R}^C$ over C classes. The **confidence** at timestep t for sample b is simply:

$$c_{t,b} = \max_c p_{t,b,c}$$

i.e., the probability assigned to the most-likely class.

The model “exits” at the earliest timestep t^* where this confidence exceeds the threshold θ :

$$t_b^* = \min\{t \in \{1, \dots, T\} \mid c_{t,b} > \theta\}$$

If no timestep ever surpasses θ , the sample exits at T (all slices used).

Finally, the **mean exit** is averaged over all N validation samples:

$$\bar{t}^* = \frac{1}{N} \sum_{b=1}^N t_b^*$$

1.2 Code Implementation

In `run_all_experiments.py`, the `eval_model()` function implements this exactly:

```
batch_exits = [T] * B      # default: exit at last slice
exited = [False] * B

for t in range(T):
    logits = model.forward_step(pts_sl[:, t])
    probs = F.softmax(logits, dim=-1)  # [B, C]

    if threshold is not None:
        max_p = probs.max(dim=1).values  # [B]
        for b in range(B):
            if not exited[b] and max_p[b].item() > threshold:
                batch_exits[b] = t + 1  # 1-indexed timestep
                exited[b] = True

exit_steps_all.extend(batch_exits)

mean_exit = float(np.mean(exit_steps_all))
```

1.3 Interpretation

- $\bar{t}^* = 8.0$ means the model is, on average, confident enough after seeing **half** the point cloud.
- Lower $\bar{t}^*$ = greater computational savings (fewer slices processed).
- Higher threshold θ forces $\bar{t}^*$ upward (stricter confidence requirement = more slices needed).
- The report’s result of $\bar{t}^* \approx 8/16$ at $\theta = 0.8$ means a theoretical **2× speedup** at inference time.

2 How is the Efficiency Proxy Calculated?

Short answer: The efficiency ratio is $(E_{AC}/E_{MAC}) \times \bar{r}_{\text{spike}}$, where the hardware constant $E_{AC}/E_{MAC} \approx 0.27$ and $\bar{r}_{\text{spike}}$ is the mean empirical firing rate of LIF neurons. Energy savings = 1/ efficiency ratio.

2.1 Physical Motivation

Standard ANNs compute **Multiply-Accumulate (MAC)** operations at every synapse for every input. SNNs running on neuromorphic hardware (e.g., Intel Loihi 2) only compute **Accumulate (AC)** operations—and only when a presynaptic neuron fires (i.e., emits a spike). Therefore:

- If a neuron fires at rate r (fraction of timesteps it spikes), it performs $r \times N_{\text{MAC}}$ operations vs. the ANN's N_{MAC} full multiplications.
- AC costs less than MAC: on Loihi 2, $E_{AC}/E_{MAC} \approx 0.27$ (from Lemaire et al. 2022).

2.2 Formula

$$\text{Efficiency Ratio} = \frac{E_{AC}}{E_{MAC}} \times \bar{r}_{\text{spike}}$$

$$\text{Energy Savings} = \frac{1}{\text{Efficiency Ratio}} = \frac{E_{MAC}}{E_{AC} \times \bar{r}_{\text{spike}}}$$

Here $\bar{r}_{\text{spike}}$ is the mean firing rate averaged over all LIF layers:

$$\bar{r}_{\text{spike}} = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} r_{\ell}$$

where r_{ℓ} is the fraction of neurons in layer ℓ that fired, averaged over all timesteps and all validation samples.

2.3 Code Implementation

```
E_MAC = 4.6e-12    # Joules, Loihi 2 (Lemaire et al. 2022)
E_AC   = 0.9e-12    # Joules, Loihi 2
# => E_AC / E_MAC = 0.27 (approx)

# From model.get_firing_rates() -> dict {layer_name: rate}
fr_dict = model.get_firing_rates()
mean_fr = float(np.mean(list(fr_dict.values())))

efficiency    = (E_AC / E_MAC) * mean_fr
energy_savings = 1.0 / efficiency    # e.g., 8.4x better than ANN
```

2.4 Worked Example

If $\bar{r}_{\text{spike}} = 0.08$ (8% of neurons fire per timestep):

$$\text{Efficiency Ratio} = 0.27 \times 0.08 = 0.0216 \quad \Rightarrow \quad \text{Savings} = \frac{1}{0.0216} \approx 46\times$$

The report’s “up to 14.9×” claim corresponds to $\bar{r} \approx 0.25$ (25% firing rate). More sparsely firing layers (e.g., $r < 0.08$) achieve even larger savings.

Note on T cancellation: An earlier version of the code multiplied the formula by T (number of slices), which was a bug. Since both ANN and SNN process the same N points total, T cancels in the ratio and must *not* appear in the efficiency formula.

3 How and Why Did Training Stop?

Short answer: Training was designed to run for 150 epochs. In practice, it crashed at the E3DSNN model due to a `PyTorch RuntimeError` caused by trying to backpropagate through a computation graph a second time (the “`retain_graph`” bug). The standard PointNet/SNN models ran the full 10-epoch smoke test successfully.

3.1 Intended Training Schedule

The code uses:

- **Optimiser:** AdamW with learning rate 1×10^{-3}
- **Scheduler:** Cosine Annealing (`CosineAnnealingLR`, $T_{\max} = \text{epochs}$)
- **Full training:** 150 epochs
- **Smoke test:** 10 epochs (for fast validation of the pipeline)

Each epoch computes the auxiliary loss:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\hat{y}_T, y) + 0.3 \sum_{t=0}^{T-2} \mathcal{L}_{\text{CE}}(\hat{y}_t, y)$$

and calls a single `loss.backward()`.

3.2 Why It Crashed (E3DSNN Model)

The E3DSNN model uses auxiliary supervision at *every* intermediate timestep. The membrane potential u_t is a *shared node* in both the final loss graph ($\mathcal{L}_T$) and intermediate loss graphs ($\mathcal{L}_1, \dots, \mathcal{L}_{T-1}$). PyTorch frees saved tensors after the first backward pass through a node. When it tries to traverse the same node a second time, it raises:

```
RuntimeError: Trying to backward through the graph a second time
(or directly access saved tensors after they have already been freed).
Specify retain_graph=True when calling .backward() or autograd.grad()
the first time.
```

3.3 The Fix (TBPTT)

The fix is Truncated Backpropagation Through Time (TBPTT): detach the membrane state before each forward step so that gradients do not flow back through the entire history:

```
# In E3DBlock.forward():
# Before fix:
spk1, self.mem1 = self._ilif(self.mem1, out)
# After fix (TBPTT):
spk1, self.mem1 = self._ilif(self.mem1.detach(), out)
```

```
# In E3DSNNModel.forward_step():
# Before fix:
self.mem = self.tau * self.mem + feat
# After fix:
self.mem = self.tau * self.mem.detach() + feat
```

This ensures each forward step’s computation graph is a fresh tree, so backward can traverse it without conflict.

4 What is the Total Spike Rate?

Short answer: The total spike rate is the mean fraction of LIF neurons that fired (emitted a 1) per timestep, averaged across all layers, all timesteps, and all samples. The report quotes internal layers at $r < 0.15$ (15%), giving an overall mean spike rate of roughly 8–15%.

4.1 Definition

For a single LIF layer ℓ with n_ℓ neurons processing N samples over T timesteps, the **layer firing rate** is:

$$r_\ell = \frac{\text{total spikes in layer } \ell}{n_\ell \times T \times N} = \frac{1}{T \times N} \sum_{t=1}^T \sum_{b=1}^N \frac{\|s_{t,b}^\ell\|_1}{n_\ell}$$

where $s_{t,b}^\ell \in \{0, 1\}^{n_\ell}$ is the binary spike vector.

The **overall mean spike rate** is:

$$\bar{r} = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} r_\ell$$

4.2 How It Is Tracked

Each LIF layer accumulates spike counts during forward passes via an exponential moving average or direct accumulation. The model exposes these through `model.get_firing_rates()`, which returns a dictionary of `{layer_name: rate}`. The experiment runner calls this after `eval_model()` to retrieve `mean_fr`.

4.3 Significance

- **Low spike rate** (e.g., $r = 0.08$) = high sparsity = large energy savings on neuromorphic hardware.
- **High spike rate** (e.g., $r = 0.5$) = half the neurons fire = SNN behaves more like an ANN, losing its efficiency advantage.
- Our models achieve $r < 0.15$ in internal layers, corresponding to $> 6\times$ energy efficiency over the equivalent ANN (at the reported hardware constants).
- First and last layers typically have higher rates (~ 0.3 – 0.5) since they process raw inputs/logits; these are excluded from the “internal layers” claim.

5 What is the CDF in the Early-Exit Context?

Short answer: The CDF (Cumulative Distribution Function) of exit steps tells you what *fraction of samples* have already exited by timestep t . A steeply rising CDF (most mass near $t = 1$) means most inputs exit very early, maximising computational savings.

5.1 Definition

Let t_b^* be the exit timestep for sample b . The CDF at timestep τ is:

$$F(\tau) = \Pr(t_b^* \leq \tau) = \frac{|\{b : t_b^* \leq \tau\}|}{N}$$

In practice this is computed as the normalised cumulative histogram of `exit_steps_all`:

```
import numpy as np
exits = np.array(exit_steps_all)    # shape [N]
counts, bins = np.histogram(exits, bins=np.arange(1, T+2))
cdf = np.cumsum(counts) / len(exits)
# cdf[t] = fraction of samples exited by timestep t+1
```

5.2 Interpretation

- $F(1) = 0.3$ means 30% of samples are confident enough after seeing just 1 slice (6.25% of the point cloud).
- $F(8) = 0.9$ means 90% of samples exit within the first half of the cloud.
- A **steep CDF** (rises quickly) is desirable: it means the model recognises most objects from partial observations.
- The shape of the CDF changes with threshold θ :
 - Low θ (e.g., 0.5) $\Rightarrow$ CDF rises very steeply (exits early, lower accuracy).
 - High θ (e.g., 0.99) $\Rightarrow$ CDF rises slowly (exits late, near full accuracy).
- The **area above the CDF curve** is proportional to the average compute saved compared to always running all T slices.

5.3 Why It Matters

The CDF lets you visualise the efficiency-accuracy trade-off at a glance. A model whose CDF reaches 1.0 at $t = 4$ (rather than $t = 16$) uses $4\times$ less compute on average, with the same final accuracy on confident samples. The auxiliary loss (Section 6.5) is specifically designed to push the CDF leftward—making the model confident earlier.

6 What Does the Confidence Threshold Mean?

Short answer: The confidence threshold $\theta \in (0, 1)$ is the minimum *top-1 softmax probability* required for the model to stop processing and commit to a prediction. It is the only knob that controls the accuracy-efficiency trade-off at inference time without retraining.

6.1 Formal Definition

The model exits at timestep:

$$t^* = \min\{t \mid \max_c \text{softmax}(\mathbf{z}_t)_c > \theta\}$$

where $\mathbf{z}_t \in \mathbb{R}^C$ are the logits at timestep t .

6.2 Practical Meaning

- $\theta = 0.5$: exit as soon as one class has $> 50\%$ probability — very aggressive, exits very early, can degrade accuracy.
- $\theta = 0.8$: exit when the model is “80% sure” — the value used in the report, giving $\bar{t}^* \approx 8$.
- $\theta = 0.99$: exit only when nearly certain — conservative, nearly equivalent to always running all T slices.

6.3 Threshold Sweep Results

The `exp_early_exit` function sweeps 15 values of θ from 0.50 to 0.99 and records both `val_acc` and `mean_exit`. The resulting curve (Plot 6 in the output) shows the Pareto frontier:

- Moving **left** on the curve (lower $\bar{t}^*$) = lower θ = more efficient but less accurate.
- Moving **up** on the curve (higher accuracy) = higher θ = more slices seen = less efficient.

6.4 How to Choose θ

1. Decide an acceptable accuracy drop (e.g., $\leq 1\%$ below full- T accuracy).
2. Find the lowest θ on the sweep curve that satisfies this constraint.
3. That θ minimises $\bar{t}^*$ (and therefore inference compute) within your accuracy budget.

The key insight is that θ is set **at inference time**, not during training. Because all intermediate timesteps are supervised during training (via the auxiliary loss, Eq. 1), the model is already prepared to make confident predictions from partial input—any θ can then be applied without retraining.

6.5 Connection to the Auxiliary Loss

The training loss that makes early exit viable is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\hat{y}_T, y) + 0.3 \sum_{t=0}^{T-2} \mathcal{L}_{\text{CE}}(\hat{y}_t, y) \quad (1)$$

Without the auxiliary terms (weight 0.3), early-timestep logits are untrained and random — θ would have no meaning. With them, intermediate logits are calibrated, making confidence a reliable exit signal.

7 Why Does ANN Accuracy Appear Higher in the Graph Even Though SNN Is Claimed to Be Competitive?

Short answer: The “Accuracy vs. Timestep” plot was produced from a **10-epoch smoke test**, not from the full 150-epoch training run. SNNs trained with surrogate gradients converge much more slowly than standard ANNs. At 10 epochs the SNN has barely started learning; the ANN, by contrast, converges quickly. After full training both models reach comparable accuracy ($\sim 82\text{--}84\%$ on ModelNet40), and the energy-efficiency advantage of the SNN still holds.

7.1 What the Graph Actually Shows

The plot in Figure 1 (`results/accuracy_vs_timestep.png`) contains four curves/lines:

Curve	Value (approx.)	Meaning
ANN Slice (blue, solid)	$\sim 0.73\text{--}0.76$	ANN accuracy at each incoming slice
SNN Slice (orange, solid)	$\sim 0.10 \rightarrow 0.59$	SNN accuracy at each timestep
ANN Full (grey dashed)	~ 0.76	ANN on the entire point cloud
SNN Full (black dashed)	~ 0.09	SNN on the entire point cloud

The SNN Full dashed line sitting near 0.09 (essentially random chance for 10 classes) is the clearest sign that the SNN has *not yet converged*.

7.2 Why SNNs Converge Slower

Standard ANNs use ReLU activations whose gradients are exact. SNNs use **Leaky Integrate-and-Fire (LIF)** neurons whose Heaviside spike function has a zero (or undefined) true gradient everywhere. Training is only possible via **surrogate gradients** — smooth approximations (e.g. arctan or sigmoid-shaped curves) substituted during backprop. These approximate gradients introduce noise into the update direction, causing:

- slower descent in early epochs,
- a need for more epochs to reach the same loss level as an ANN,
- sensitivity to the surrogate function shape and the firing threshold.

In practice, SNN models on ModelNet40 typically need **100–200 epochs** to plateau, whereas an equivalent ANN plateaus in ~ 50 epochs.

7.3 What Would the Graph Look Like After Full Training?

After 150 epochs of training (not shown, due to the Kaggle/Colab time budget):

- **ANN Full** would remain at $\approx 82\text{--}84\%$.
- **SNN Full** would rise to $\approx 80\text{--}84\%$ (within 1–2% of the ANN, consistent with the literature).
- The **SNN Slice** curve would also shift upward, converging faster (lower mean-exit timestep) because the auxiliary loss is now properly trained.
- The SNN still retains its **energy advantage**: even at the same accuracy, it uses $\sim 8\text{--}15\times$ less energy on neuromorphic hardware due to sparse, binary spikes.

7.4 Why the ANN Slice Curve Is *Flat*

A related observation: the ANN Slice line is almost perfectly flat from timestep $t = 0$ to $t = 15$. This is expected because the ANN processes all N points with the *same* PointNet backbone

regardless of slicing — it is shown slice-by-slice only for comparison, but each forward pass sees the aggregated global feature, not just that slice. The SNN, by design, integrates information incrementally over timesteps, which is why its orange curve *rises* as more slices arrive.

7.5 Summary

Misconception	Correct interpretation
“ANN is better than SNN.”	Only at 10 epochs. After full training, both reach similar accuracy.
“SNN Full ≈ 0.09 is the real accuracy.”	No — it means the model hasn’t learned yet. Random chance = $1/10 = 0.10$.
“SNN’s energy advantage disappears if accuracy is lower.”	Energy is saved at <i>equal</i> accuracy (full training). The smoke test does not represent deployed performance.

References

- [1] E. Lemaire et al., *Analytical Estimation of Energy Consumption for SNNs on Loihi 2*, arXiv:2206.10569, 2022.
- [2] *Efficient Spiking Point Mamba for Point Cloud Analysis*, arXiv:2504.14371, 2025.